Etapa 2 — Subindo o Banco de Dados com Docker

1. Criar a rede Docker

```
docker network create todo-network
```

2. Criar e executar o container PostgreSQL

```
docker run -d `
  --name todo-postgres `
  --network todo-network `
  -e POSTGRES_USER=postgres `
  -e POSTGRES_PASSWORD=SUA_SENHA_REAL `
  -e POSTGRES_DB=meubackend `
  -p 5432:5432 `
  postgres:16-alpine
```

3. Criar e executar o container do backend

```
docker run -d `
  --name todo-backend `
  --network todo-network `
  -e DB_HOST=todo-postgres `
  -e DB_PORT=5432 `
  -e DB_USER=postgres `
  -e DB_PASSWORD=SUA_SENHA_REAL `
  -e DB_NAME=meubackend `
  -p 8000:8000 `
  meubackend
```

4. Verificar os containers em execução

```
docker ps
```

5. Verificar os logs do backend

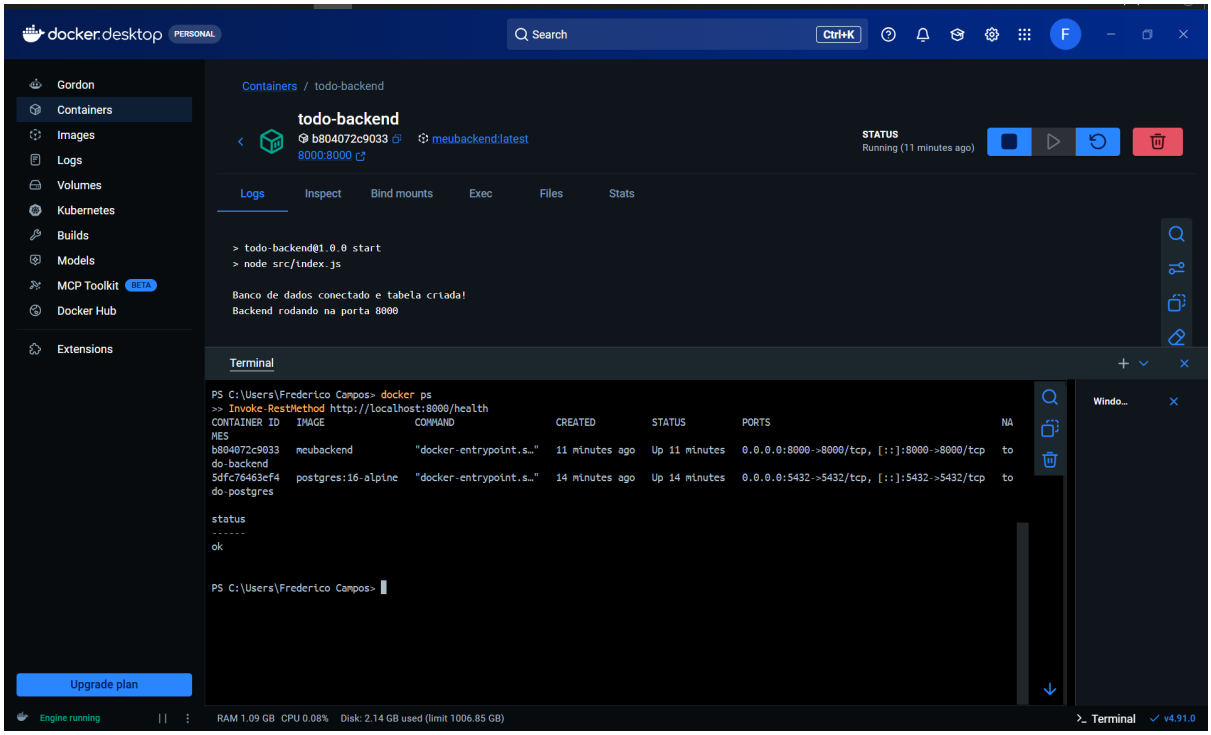
```
docker logs todo-backend
```

6. Verificar a conexão com o PostgreSQL

```
docker exec todo-postgres pg_isready -U postgres -d meubackend
```

7. Testar o endpoint de saúde da API

```
Invoke-RestMethod http://localhost:8000/health
```


docker desktop
PERSONAL
Search
Ctrl+K
Gordon
Containers
Images
Logs
Volumes
Kubernetes
Builds
Models
MCP Toolkit
BETA
Docker Hub
Extensions
Upgrade plan
Engine running
Containers / todo-backend
todo-backend
b804072c9033
meubackend:latest
8000:8000
STATUS
Running (11 minutes ago)
Logs
Inspect
Bind mounts
Exec
Files
Stats
> todo-backend@1.0.0 start
> node src/index.js
Banco de dados conectado e tabela criada!
Backend rodando na porta 8000
Terminal
PS C:\Users\Frederico Campos> docker ps
>> Invoke-RestMethod http://localhost:8000/health
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NA
MES
b804072c9033 meubackend "docker-entrypoint.s…" 11 minutes ago Up 11 minutes 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp to
do-backend
5dfc76463ef4 postgres:16-alpine "docker-entrypoint.s…" 14 minutes ago Up 14 minutes 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp to
do-postgres
status
ok
PS C:\Users\Frederico Campos>
Windo...
RAM 1.09 GB CPU 0.08% Disk: 2.14 GB used (limit 1006.85 GB)
Terminal
v4.91.0